

3.1 Регресія. Лінійна регресія

У цьому розділі розглядаються фундаментальні алгоритми для прогнозування безперервних величин, методи боротьби з перенавчанням та розширення лінійних моделей для роботи з нелінійними даними.

Перед початком роботи з машинним навчанням необхідно підключити бібліотеки,

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.metrics import r2_score
```

які відповідають за обчислення, обробку даних, побудову графіків та створення моделей:

- NumPy – використовується для швидких математичних операцій, генерації чисел, роботи з матрицями та векторизації;
- Pandas – бібліотека для роботи з таблицями та аналізу даних;
- Matplotlib – використовується для побудови графіків та візуалізації даних;
- **train_test_split** – використовується для поділу даних на тренувальну вибірку (train) та тестову вибірку (test). Це необхідно для перевірки якості моделі на нових даних, які вона ще не бачила;
- **fetch_california_housing** – готовий набір даних з бібліотеки scikit-learn. Він містить інформацію про житло в Каліфорнії;
- **LinearRegression** – базова модель лінійно регресії, яка намагається провести найкращу пряму через дані. Використовується для прогнозування безперервних величин: ціна, температура, кількість продажів тощо;
- **Ridge** – розширення лінійної регресії з L2-регуляризацією. Регуляризація додає обмеження за занадто великі коефіцієнти моделі, що допомагає боротися з перенавчанням;
- **Lasso** – модель з L1-регуляризацією. На відміну від Ridge, може зводити деякі коефіцієнти до нуля, фактично виконуючи автоматичний відбір ознак;

- **PolynomialFeatures** – створює поліноміальні ознаки ($x \rightarrow x^2, x^3$). Це дозволяє лінійній моделі працювати з нелінійними залежностями;
- **StandardScaler** – використовує стандартизацію даних, де 0 – середнє значення, 1 – стандартне відхилення. Масштабування важливе для моделей з регуляризациєю;
- **r2_score** – метрика оцінки якості регресії (наскільки добре модель пояснює залежність у даних), де 1 – ідеальний прогноз, 0 – модель не краща за випадкове вгадування.

Основи лінійної регресії та підготовка даних

Перед побудовою моделі необхідно підготувати дані та розділити їх на окремі вибірки. Це дозволяє чесно оцінити якість алгоритму.

```
def load_data():  
    data = fetch_california_housing()  
    X = pd.DataFrame(data.data, columns=data.feature_names)  
    y = data.target  
  
    return train_test_split(X, y, test_size=0.2, random_state=42)  
  
X_train, X_test, y_train, y_test = load_data()
```

Функція `fetch_california_housing()` завантажує готовий датасет з бібліотеки `scikit-learn`. Він містить інформацію про райони Каліфорнії: середній дохід, кількість кімнат, населення, географічне розташування тощо.

У задачах машинного навчання дані зазвичай поділяються на: **ознаки (features)** – вхідні параметри; **цільову змінну (target)** – те, що потрібно передбачити.

```
X = pd.DataFrame(data.data, columns=data.feature_names)  
y = data.target
```

X – таблиця з характеристиками будинків, y – масив цін житла, `pd.DataFrame()` використовується для перетворення даних у зручну табличну структуру `pandas`.

Для об'єктивної оцінки модель не можна перевіряти на тих самих даних, на яких вона навчалась. Саме тому використовується `train_test_split()`.

```
train_test_split(X, y, test_size=0.2, random_state=42)
```

Функція автоматично ділить дані на дві частини: **train** – використовується для навчання моделі; **test** – використовується для перевірки якості. Параметр `test_size=0.2` означає, що 80% даних піде на навчання, а 20% на тестування. `random_state=42` фіксує випадковість, щоб при кожному запуску поділ був однаковим.

При лінійній регресії модель намагається знайти лінійну залежність між ознаками та результатом:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

де:

- x – ознаки (дохід, вік будинків, кількість кімнат),
- w – коефіцієнти моделі (показують, наскільки важлива кожна ознака),
- b – зсув, базове значення (стартова ціна будинку),
- y – прогнозоване значення (ціна будинку).

Класична лінійна регресія використовує метод найменших квадратів (Ordinary Least Squares, OLS). Мета алгоритму – підібрати такі коефіцієнти, щоб різниця між реальними та прогнозованими значеннями була мінімальною. Для цього мінімізується сума квадратів помилок:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

де:

- y_i – справжнє значення цільової змінної,
- $\hat{y}_i$ – прогнозування моделі,
- n – кількість прикладів.

Чим менше значення цієї функції, тим точніше модель описує дані.

Після навчання модель зберігає коефіцієнти (w) та вільний член (b). Коефіцієнти показують, наскільки сильно певна ознака впливає на результат: **додатне значення** – збільшення ознаки підвищує прогноз; **від'ємне** – знижує. Наприклад для одного району модель візьме значення з таблиці, і самостійно підібрані нею ваги:

$$y = 0.7 \cdot MedInc + 0.02 \cdot HouseAge + 0.1 \cdot AveRooms + 0.0001 \cdot Population + b$$

$$y = 0.7 \cdot 3 + 0.02 \cdot 30 + 0.1 \cdot 5 + 0.0001 \cdot 0.5 = 3.9$$

Тобто за прогнозом моделі будинок в цьому районі буде вартувати $\approx 390\,000$ \$. Вільний член b - це просто базове значення, з якого модель починає рахувати. Наприклад, якщо район порожній (всі $x = 0$), модель все одно дає якусь базову ціну.

$$y = 0 + 0 + 0.5 = 0.5$$

Тобто, $\approx 50\,000$ \$ це стартова ціна будинку. Без цього коефіцієнту всі прогнози завжди починались би з нуля, модель була би прив'язана до нуля і як наслідок дуже обмежена.

3.1.1 Регуляризація. Ridge та Lasso

Звичайна лінійна регресія добре працює на простих даних, однак при великій кількості ознак модель може перенавчатись. У такому випадку вона занадто точно запам'ятовує тренувальні дані та гірше працює на нових прикладах. Для боротьби з цією проблемою використовують регуляризацію - додаткове обмеження на коефіцієнти моделі.

```
models = {  
    "Linear Regression": LinearRegression(),  
    "Ridge (L2)": Ridge(alpha=1.0),  
    "Lasso (L1)": Lasso(alpha=0.1)  
}  
  
print("--- Оцінка базових моделей (R2 Score) ---")  
for name, model in models.items():  
    model.fit(X_train, y_train)  
    train_score = model.score(X_train, y_train)  
    test_score = model.score(X_test, y_test)  
    print(f"{name:18}: Train={train_score:.4f}, Test={test_score:.4f}")
```

Метод `fit()` запускає процес навчання моделі на тренувальних даних. Після цього модель підбирає оптимальні коефіцієнти для прогнозування цільової змінної.

Метод `score()` для регресії повертає коефіцієнти детермінації R^2 . Якщо R^2 рівне: 1.0 - ідеальна модель; 0.7-0.8 - добре працює; 0.4-0.6 - середня модель; 0 або менше - майже не працює.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Як це читати? Якщо `train=0.75` - модель добре пояснює дані, на яких навчається; `test=0.70` - як вона працює на нових даних. Велика різниця між тренувальною та тестовою вибірками часто свідчить про перенавчання моделі.

Ridge

Ridge Regression додає штраф за великі коефіцієнти моделі (L2-регуляризація).

$$\sum (y_i - \hat{y}_i)^2 + \alpha \sum w_i^2$$

де:

- перша частина - помилка моделі,
- друга частина - обмеження за великі ваги.

Параметр α (альфа) контролює силу регуляризації: велике значення - сильніше обмеження коефіцієнтів, маленьке - поведінка ближча до звичайної регресії.

Ridge особливо корисний при мультиколінеарності - ситуації, коли ознаки сильно пов'язані між собою. Простіше, коли дві або більше ознак майже дублюють одна одну (площа будинку і кількість кімнат).

Lasso

Lasso Regression використовує інший тип штрафу - L1-регуляризацію.

$$\sum (y_i - \hat{y}_i)^2 + \alpha \sum |w_i|$$

Особливість Lasso полягає в тому, що деякі коефіцієнти можуть ставати рівними нулю. Це означає, що модель: автоматично відкидає неважливі ознаки, виконує спрощення моделі, фактично проводить відбір ознак (features).

В коді ми одночасно перевірили: звичайну лінійну регресію, Ridge та Lasso. Такий підхід дозволяє оцінити: чи покращує регуляризація якість, наскільки модель схильна до перенавчання, чи є зайві ознаки у даних.

Які висновки можна зробити тут:

```
--- Оцінка базових моделей (R2 Score) ---  
Linear Regression : Train=0.6126, Test=0.5758  
Ridge (L2)       : Train=0.6126, Test=0.5759  
Lasso (L1)       : Train=0.5489, Test=0.5318
```

- Linear Regression та Ridge майже однакові – це означає, що дані досить стабільні і сильного перенавчання немає.
- Train та Test результати близькі, модель добре узагальнює та запам'ятовує дані.
- Lasso гірший по якості – це означає, що зайвих ознак майже немає і всі змінні несуть корисну інформацію.

3.1.2 Математичні підходи. Нормальне рівняння та Поліноміальна регресія

Лінійна регресія може бути реалізована не лише через готові моделі `scikit-learn`, а й через математичні формули та матричні операції. Крім того, лінійні моделі можна розширювати для роботи з нелінійними залежностями за допомогою поліноміальних ознак.

Нормальне рівняння (Normal Equation)

Один із класичних способів знаходження параметрів лінійної регресії є аналітичне розв'язання через нормальне рівняння.

```
def solve_normal_equation(X, y):  
    X_b = np.c_[np.ones((X.shape[0], 1)), X]  
  
    theta_best = np.linalg.pinv(X_b.T @1 X_b) @ X_b.T @ y  
  
    return theta_best  
  
print("--- Матричне розв'язання (Normal Equation) ---")  
  
theta = solve_normal_equation(X_train.values, y_train)  
  
print(f"Intercept (b): {theta[0]:.4f}")  
print(f"Перші 3 ваги (w): {theta[1:4]}")
```

Перед обчисленням створюється нова матриця `X_b`:

```
X_b = np.c_[np.ones((X.shape[0], 1)), X]
```

¹ @ - оператор матричного множення

Стовпець одиниць додається для врахування вільного члена b у рівнянні регресії. Параметри моделі обчислюються за формулою:

$$\theta = (X^T X)^{-1} X^T y$$

де:

- X - матриця ознак,
- y - вектор відповідей,
- θ - параметри моделі.

У коді використовується `np.linalg.pinv()` - псевдообернена матриця. Вона стабільніша за звичайне обернення матриці та дозволяє працювати навіть тоді, коли матриця містить зайву повторювану інформацію. Результатом є вектор `theta`:

```
theta = [b, w1, w2, w3, w4, w5, w6, w7, w8]
```

`theta[0]` - вільний член b ; решта значень - ваги w (кількість ознак (колонок)).

```
--- Матричне розв'язання (Normal Equation) ---  
Intercept (b): -37.0233  
Перші 3 ваги (w): [ 0.44867491  0.00972426 -0.12332334]
```

Від'ємний Intercept (b) сам по собі не є ні добре, ні погано - це нормально для лінійної регресії. Цей коефіцієнт є просто зсувом всієї моделі вгору або вниз, головне щоб не значення не дорівнювало чистому нулю.

Ваги - це коефіцієнти, які показують, наскільки кожна ознака впливає на прогноз моделі. Позитивні значення означають, що зі збільшенням відповідної ознаки прогноз зростає (чим вищий середній дохід у районі - тим вища ціна житла), а від'ємні - що зменшується.

Поліноміальна регресія (Polynomial Regression)

Звичайна лінійна регресія може погано працювати на нелінійних залежностях². Щоб зробити модель гнучкішою, використовують поліноміальні ознаки.

² Нелінійні залежності - зв'язки між змінними, які не можна описати прямою лінією. Наприклад, залежність ціни житла від кількості кімнат - після певної кількості кімнат ціна перестає суттєво зростати, тобто на графіку цей зв'язок більше б нагадував гіперболу, ніж пряму лінію.

```
print("--- Поліноміальна регресія та масштабування (Polynomial Regression and Scaling) ---")

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_poly)
X_test_scaled = scaler.transform(X_test_poly)

poly_model = Ridge(alpha=100)
poly_model.fit(X_train_scaled, y_train)

print(f"Poly Train Score: {poly_model.score(X_train_scaled, y_train):.4f}")
print(f"Poly Test Score : {poly_model.score(X_test_scaled, y_test):.4f}")
```

Модуль `PolynomialFeatures` створює нові ознаки на основі степенів та комбінацій існуючих.

```
poly = PolynomialFeatures(degree=2, include_bias=False)
```

При `degree=2` модель автоматично створює: x^2 , xy , y^2 тощо. Наприклад:

$$y = w_1x + w_2x^2 + b$$

Таким чином лінійна модель може описувати криві та складніші залежності.

Після створення поліноміальних ознак значення можуть сильно відрізнятися за масштабом. Через це регуляризація починає працювати некоректно. Для вирішення проблеми застосовується `StandardScaler()`, який стандартизує дані³: центрує їх навколо 0, робить стандартне відхилення рівним 1.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_poly)
X_test_scaled = scaler.transform(X_test_poly)
```

Без масштабування великі ознаки можуть почати домінувати над малими, регуляризація працюватиме нестабільно і модель може гірше навчитись. Особливо це критично для

³ У реальних даних ознаки можуть мати дуже різні масштаби, наприклад: середня зарплата, вартість квартири, кількість кімнат. Без стандартизації великі значення можуть домінувати над іншими ознаками. Це відбувається через те що комп'ютер не розуміє контексту і просто аналізує числа, що може сповільнити навчання та погіршити якість моделі

алгоритмів на основі відстаней. Після створення нових ознак використовується Ridge-регресія:

```
poly_model = Ridge(alpha=100)
```

Регуляризація тут особливо важлива, оскільки поліноміальні ознаки різко збільшують складність моделі та ризик перенавчання.